

Железо для ИИ: от задачи к конфигурации

Аппаратная конфигурация отвечает на вопрос о конкретной работе. Для одной документной системы вместимость модели, задержка ответа и возможность дообучения оказываются разными решениями.

Не одна «машина для ИИ», а три обещания

Представим локальную систему для исследовательской команды: около трёх тысяч PDF и таблиц, после обработки условно 150 тысяч фрагментов, пятнадцать пользователей. Поиск должен возвращать ответ с точной страницей либо диапазоном ячеек. В учебном опыте двадцать запросов могут прийти одновременно; это ещё не означает двадцать одновременно исполняемых генераций. Команда хотела бы получать полный ответ не позднее двух минут, а затем иногда дообучать модель на собственных примерах. Эти числа задают сценарий, а не измерение чужой или существующей у команды системы. Сначала нужно разделить три обещания. *Запуск* — выбранный checkpoint загружается и отвечает на один запрос. *Обслуживание* — при требуемой нагрузке выдержаны задержка и качество ссылок. *Обучение* — система выполняет шаг обновления весов и сохраняет воспроизводимый результат. У каждого режима свои массивы памяти, очередь и цена ошибки. Успех первого не доказывает второго или третьего.

Архитектура как распределение работы

Документы читаются с SSD, парсер и OCR создают текст с привязкой к страницам, процесс поиска отбирает фрагменты, а модель получает только нужную часть корпуса. CPU, RAM, накопитель, GPU и сеть участвуют в разных стадиях; их нельзя заменить одной цифрой FLOPS. CUDA Programming Guide¹ различает код ведущего CPU и исполнение на GPU. Best Practices² показывает, почему движение данных между ними следует измерять отдельно. Сервер, у которого матричные операции быстры, всё равно может ждать парсер, индекс или очередь.

Сквозной вопрос выпуска

Сможет ли машина с 6 GiB видеопамяти поддержать этот процесс и когда понадобится другой класс устройства? Для расчётов введём условный плотный decoder-only генератор с 8 млрд параметров, 32 слоями, восемью KV-головами размером 128 и удерживаемой длиной последовательности 4096 токенов. Эти параметры вымышлены и не являются паспортом конкретной модели. Сначала выясним, где находятся данные, затем посчитаем нижнюю границу памяти, сопоставим её с очередью и оценим обучение, индекс и локальный запуск. Финальное решение оставим зависимым от фактического теста качества и скорости.

Три исхода одной задачи

1. Запустить

Один запрос, точный checkpoint и фактический пик памяти.

2. Обслужить

Двадцать поступивших запросов; сколько генераций одновременно держать в GPU и сколько оставлять в очереди?

3. Изменить

Полное обновление весов или адаптер: какие состояния обучения нужны?

С чего начать чтение

CUDA Programming Guide¹, Programming Model и Understanding Memory: проследить, где лежат данные одного запроса и кто перемещает их.

Граница выпуска

Все размеры и времена ниже учебные. Требования к конкретному checkpoint и устройству подтверждаются только пробным запуском.

Подтемы главы

1. Путь данных и узкое место
2. Веса, KV и конкурентность
3. Шаг обучения и кластер
4. Очередь и инференс
5. Корпус, индекс и копии
6. Локальная модель
7. Выбор и проверка машины

Паспорт нагрузки как состояние решения

Чтобы не подменять одно обещание другим, зафиксируем учебный паспорт $\mathcal{W} = (D, N, P, S, A, C, T, Q)$. Здесь D — исходные документы, N — число фрагментов, P — параметры генератора, S — токены удерживаемой последовательности, A — запросы, уже поступившие в систему, C — одновременно исполняемые последовательности, T — ограничение времени ответа, Q — проверка качества цитат. Это авторская схема учёта, не математическая модель всей системы. В частности, $A = 20$ не требует $C = 20$: меньший C экономит KV-память, но добавляет ожидание.

Из паспорта получаются проверяемые вопросы. Если P и S известны, можно оценить сырые веса и кэш, но нельзя узнать пик выделенной VRAM без runtime. Если C ограничен, надо измерить очередь при одновременном приходе A запросов. Если D изменяется, для ссылок необходима связь от ответа к версии документа. Для обучения к тем же весам добавятся градиенты, оптимизатор и активации. Одной формулой все четыре случая не описываются.

Вычислительный потолок и сервис

В Roofline³ для одной операции вычислительная производительность ограничивается $\min(P_{\max}, IB_{\max})$, где I — число операций на байт перемещения, $B_{\max}$ — пропускная способность памяти, $P_{\max}$ — пик подходящих вычислений. Это верхняя граница для ядра, а не предсказание времени ответа с поиском и очередью. FlashAttention⁴ демонстрирует изменение пути данных алгоритмом: меньше промежуточных переносов между уровнями памяти при сохранении точного attention. Значит, число операций и паспортный пик GPU без профиля нагрузки не дают ранжирования покупок.

Документ решения

В протоколе нужны ревизия модели и tokenizer, формат весов, размеры входов и выходов, число поступивших и одновременно обслуживаемых запросов, версия стека, пик памяти, задержка, качество ответов и стоимость. Эти поля будут заполнены по этапам. Вводное решение пока скромное: имеющаяся машина годится для исследовательского прототипа лишь после опыта на точном checkpoint; для многопользовательского сервиса и обучения право на вывод придётся получить отдельно.

Учебный паспорт

Корпус

$D \approx 3000$ PDF/Excel;

$N = 150\,000$ фрагментов.

Размер файлов и индекса пока неизвестен.

Модель

$P = 8$ млрд, $S = 4096$,

параметры attention заданы только для расчёта.

Нагрузка

$A = 20$ пришедших; $C = 1, 4, 20$

— три режима для проверки, а не три наблюдения.

Условие

$T \leq 120$ s вместе с поиском; Q требует точной страницы или диапазона ячеек.

Где разобрано

Roofline³: operational intensity и две верхние границы.

FlashAttention⁴: какие промежуточные данные перестают записываться во внешнюю память GPU.

Проверить различие

Если C уменьшить с двадцати до четырёх, какая часть паспорта сократит память, а какая увеличит ожидание?

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. NVIDIA (документация, 2026), *CUDA Programming Guide*. Документация.

Руководство по программированию CUDA

Описаны устройство вычислительных блоков, пространства памяти, запуск ядер и программная модель CUDA. Термины помогают отделить локальность данных от паспортного числа операций в секунду.

Что читать. Programming Model; Understanding Memory; Features and Technical Specifications. Нарисовать путь данных от RAM до регистров.

2. NVIDIA (документация, 2026), *CUDA C++ Best Practices Guide*. Документация.

Практическое руководство по производительности CUDA

Документация разбирает передачу данных, локальность, coalescing и профилирование. Полезна для понимания, почему фактическая пропускная способность ниже рекламируемой и когда загрузка GPU не означает эффективную работу.

Что читать. Data Transfer Between Host and Device; Memory Optimizations; Performance Metrics. Сравнить requested и actual bandwidth.

3. Williams, S.; Waterman, A.; Patterson, D. (2009), *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures*. Исследовательская статья.

Roofline: модель верхних границ вычислений и обмена с памятью

Связывает достижимую производительность ядра с арифметической интенсивностью, пиком вычислений и пропускной способностью памяти. Даёт язык для гипотезы об узком месте, но не прогнозирует задержку целого сервиса.

Что читать. Разделы о модели и operational intensity; построить две границы для одной операции, указать допущения.

4. Dao, T. и др. (2022), *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS.

FlashAttention: точное внимание с учётом обмена между уровнями памяти

Разбиение вычисления attention на блоки уменьшает движение данных между HBM и быстрой памятью GPU. Это пример ускорения без изменения математической цели attention; эффект зависит от формы задачи и реализации.

Что читать. Постановка IO-aware attention и раздел Algorithm; объяснить, какие промежуточные данные перестают записываться в HBM.

Порядок чтения. Сначала CUDA и Best Practices задают словарь устройств и передач. Затем Roofline объясняет потолок отдельной операции, FlashAttention показывает алгоритмическую альтернативу увеличению FLOPS. После чтения заполнить свой паспорт нагрузки; ни одна работа не обещает скорость полного сервиса.

1. Путь данных и узкое место

Один ответ проходит через документы, поиск и генератор. Выбор компонента начинается с трассы этого пути и проверяемого объяснения задержки.

След одного ответа

Пользователь спрашивает, что сказано в таблице на странице 7 обновлённого PDF. Запрос получает идентификатор; процесс поиска читает метаданные, находит фрагмент и возвращает текст вместе с версией документа и координатой. CPU выполняет управление, токенизацию и часть фильтров, RAM держит рабочие страницы и индекс, SSD хранит оригинал и checkpoint. GPU исполняет embedding или reranker, если для них выбран такой backend, а затем prefill и decode генератора. Ответ уходит пользователю через сетевой процесс. Это причинная цепь, а не утверждение, что все стадии всегда занимают одинаковое время.

Programming Model¹ объясняет границу host и device: команда CPU запускает ядра и управляет передачами. Даже внутри GPU данные переходят между глобальной и быстрой памятью на чипе. Ёмкость VRAM отвечает на вопрос, помещается ли рабочее множество; bandwidth и локальность — насколько быстро подаются данные; поддерживаемые операции и вычислительные блоки — что удаётся сделать с ними. Эти вопросы надо разделять до сравнения устройств.

Где хранятся разные состояния

Исходные PDF и Excel, OCR-текст, векторы, журнал изменения и резервные копии — долговременные данные; они не обязаны занимать VRAM. Загруженные веса и KV активных последовательностей — рабочее состояние генератора. RAM может содержать файловый кэш и часть весов при выгрузке, но это не тот же путь доступа, что локальная память GPU. Big Model Inference³ позволяет разместить слои по устройствам; возможность загрузить большой checkpoint через offload не равна приемлемой скорости.

Гипотеза об узком месте

Пусть первый ответ медленный. Возможны разные причины: парсер ещё строит корпус, индекс выдаёт мало кандидатов, генератор долго обрабатывает длинный вход или запрос стоит в очереди. Покупка GPU проверяет лишь часть объяснений. Для каждой стадии записываем вход, выход и время: версия фрагмента; момент завершения поиска; начало и конец prefill; время первого и последующих токенов. Только после такой трассы можно назначить компонент, который действительно ограничивает задачу.

Трасса одного вопроса

Вход

PDF v2, с. 7; вопрос пользователя и ID запроса.

Поиск

Фрагмент + версия + страница из базы в RAM/SSD.

Модель

После токенизации входят фрагмент и вопрос; веса и KV читаются с GPU либо частично выгружаются.

Выход

Ответ со ссылкой на тот же PDF v2 и замерами по стадиям.

Адрес чтения

Programming Model¹: host, device, копирование и запуск ядра. Big Model Inference³: device map и следствие выгрузки.

Различающий вопрос

Если поиск занимает большую часть времени до prefill, что может изменить более быстрая видеокарта?

Цена переноса вместо абстрактной скорости

Best Practices Guide² предлагает измерять реальные host-device передачи и эффективность доступа к памяти. Для объёма D и фактически доступной полосы B_{eff} время одной передачи не может быть меньше D/B_{eff} . Представим условное чтение 4 GB: при 400 GB/s нижняя граница 0,01 s, при 16 GB/s — 0,25 s. Это сравнение двух заданных каналов, а не скорости какого-либо GPU или обещание 25-кратного ускорения ответа. Реальные ядра читают данные многократно и по-разному; offload может исполнять слой на CPU, а не копировать все веса перед каждым токеном.

Roofline⁴ предлагает ещё один различающий вопрос: сколько полезных операций приходится на каждый перемещённый байт? При обработке длинного prompt prefill может использовать крупные матричные операции. При одиночном decode интенсивность и загрузка блоков часто другие. Квантизация, batch и реализация внимания меняют границу. Поэтому профилируем prefill и decode по отдельности, фиксируя форму запроса, а не переносим одну цифру FLOPS на весь процесс.

Физический узел как ограничение маршрута

Если выбран дискретный GPU, проверяем PCIe-разъём и линии, размеры карты, питание и возможность длительного охлаждения. CPU нужен для парсинга, очереди и поиска; важны работа одного потока и параллельных задач, поддерживаемые инструкции и каналы RAM. SSD оценивают по корпусу, checkpoint, временным файлам, журналу и восстановлению, а не одной пиковой последовательной скорости. Для сервера добавляются удалённый доступ и путь замены диска. Но до измерения трассы нельзя сказать, который из этих параметров ограничит наш запрос.

Первый опыт без покупки

На одной версии корпуса и модели измерьте короткий и длинный вопрос после прогрева. Сохраните время поиска, TTFT, устойчивую скорость decode, занятость CPU, GPU, RAM и SSD. Затем измените только размещение весов: полностью на GPU или допустимая выгрузка в RAM. Если общая задержка заметно меняется, проверяйте перенос и место исполнения; если доминирует поиск, переходите к индексу. Следующий раздел посчитает, какие режимы вообще могут остаться в памяти.

Условная передача

Один объём

$D = 4$ GB. Это похоже на нижнюю оценку квантованных весов 8B.

Быстрый путь

При $B_{\text{eff}} = 400$ GB/s получаем $D/B_{\text{eff}} = 0,01$ s.

Узкий путь

При 16 GB/s получаем 0,25 s. Ни одно число не равно времени токена.

Проверка гипотезы

Best Practices², Data Transfer Between Host and Device: измерить реальный обмен. Roofline⁴: отделить предел ядра от задержки сервиса.

Что изменит решение

Две одинаковые нагрузки, один checkpoint; отличается только размещение весов. Записать время передачи и общую задержку.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. NVIDIA (документация, 2026), *CUDA Programming Guide*. Документация.

Руководство по программированию CUDA

Описаны устройство вычислительных блоков, пространства памяти, запуск ядер и программная модель CUDA. Термины помогают отделить локальность данных от паспортного числа операций в секунду.

Что читать. Programming Model; Understanding Memory; Features and Technical Specifications. Нарисовать путь данных от RAM до регистров.

2. NVIDIA (документация, 2026), *CUDA C++ Best Practices Guide*. Документация.

Практическое руководство по производительности CUDA

Документация разбирает передачу данных, локальность, coalescing и профилирование. Полезна для понимания, почему фактическая пропускная способность ниже рекламируемой и когда загрузка GPU не означает эффективную работу.

Что читать. Data Transfer Between Host and Device; Memory Optimizations; Performance Metrics. Сравнить requested и actual bandwidth.

3. Hugging Face (документация, 2026), *Big Model Inference*. Документация.

Инференс моделей, не помещающихся целиком в ускоритель

Разъясняет dispatch весов между GPU, RAM и диском. Такая выгрузка может сделать запуск возможным, но пропускная способность пути передачи меняет практическую скорость.

Что читать. Big Model Inference и описание device map; различить факт загрузки и измеренную скорость.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

4. Williams, S.; Waterman, A.; Patterson, D. (2009), *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures*. Исследовательская статья.

Roofline: модель верхних границ вычислений и обмена с памятью

Связывает достижимую производительность ядра с арифметической интенсивностью, пиком вычислений и пропускной способностью памяти. Даёт язык для гипотезы об узком месте, но не прогнозирует задержку целого сервиса.

Что читать. Разделы о модели и operational intensity; построить две границы для одной операции, указать допущения.

Порядок чтения. CUDA показывает, кто управляет исполнением и где память. Затем Best Practices учит замерять передачи; Big Model Inference показывает допустимый offload; Roofline объясняет, когда данные ограничивают отдельную операцию. После чтения построить трассу собственного запроса и проверить одну альтернативную причину задержки.

2. Веса, KV и конкурентность

Помещение checkpoint в VRAM — лишь первая ступень. Здесь считаем, сколько памяти добавит каждый активный ответ и почему число пришедших запросов не равно размеру batch.

Сырые веса: полезная, но неполная граница

Наш условный генератор содержит 8 млрд параметров. При идеальных 4 bit на параметр исходный массив весов занимает $8 \cdot 10^9 \cdot 4/8 = 4 \cdot 10^9$ bytes, или около 3,73 GiB. Формат файла обычно хранит также масштабы, служебные данные и иногда тензоры другой точности. Это не обещание, что файл или модель займёт ровно 3,73 GiB. Quantizing models in llama.cpp⁴ помогает отличить битность элементов от фактического размера выбранного файла; качество после квантования следует проверить на своих вопросах и цитатах.

Для 6 GiB VRAM от грубой границы осталось бы 2,27 GiB. Туда ещё попадут KV-кэш, временные буферы, рабочие области ядер, фрагментация и, возможно, второй генератор или embedding. В таблице Quantizing models in llama.cpp⁴, например, файл *другой* конкретной модели Llama 3.1 8B в формате Q4_K_M указан как 4,58 GiB. Нельзя подставлять его параметры в нашу формулу KV, но разница с идеальным Q4 хорошо показывает необходимость смотреть фактический файл. RAM может разгрузить GPU, как показывает Big Model Inference², но тогда следующий опыт должен измерить скорость выбранного пути.

Размер одной удерживаемой последовательности

Для самовнимания с групповыми KV-головами сырые элементы кэша на C последовательностей равны $2LH_{KV}dSC$. Двойка означает key и value, L — число слоёв, H_{KV} — KV-голов, d — размер головы, S — реально удерживаемые токены. При 32 слоях, 8 головах, 128 измерениях, 4096 токенах и 2 bytes на элемент получается $2 \cdot 32 \cdot 8 \cdot 128 \cdot 4096 \cdot 2 = 536\,870\,912$ bytes, то есть 0,5 GiB на полностью заполненную последовательность. Это верхняя точка именно для указанной длины; динамическое выделение, скользящее окно, иной KV-формат и разделение префиксов изменят фактический пик. Caching¹ описывает несколько реализаций, а не одну универсальную форму кэша.

Вопрос для закупки

Если двадцать запросов уже поступило, сколько держать активными? При $C = 1$ нижняя сумма весов и KV равна 4,23 GiB; при четырёх — 5,73; при двадцати — 13,73. К этим значениям добавляются все ещё неизвестные расходы runtime. Значит, 6 GiB не даёт запаса даже для режима $C = 4$; 24 GiB оставляют условно 10,27 GiB при $C = 20$, но лишь до накладных расходов и других моделей. Решение пока состоит в списке режимов для фактического теста, не в покупке «24 GiB по формуле».

Что именно считали

Веса

8B при идеальном Q4: 4 GB
 $\approx 3,73$ GiB.

KV

0,5 GiB на одну полностью заполненную последовательность в этом примере.

Сумма

$C = 1$: 4,23, $C = 4$: 5,73,
 $C = 20$: 13,73 GiB — без runtime.

Условие изменения

Если checkpoint имеет иные L, H_{KV}, d или длину S , пересчитать формулу. Если формат KV иной, заменить bytes на элемент.

Прочитать и проверить

Caching¹: способ выделения и offload. Quantizing⁴: реальный размер файла того же checkpoint в нескольких форматах.

Нижняя оценка против реального пика

PagedAttention³ показывает, что обслуживание многих последовательностей упирается не только в число байтов, но и в выделение переменных по длине блоков. Для одного запроса на 1000 токенов и другого на 4096 статическое резервирование полного окна даст другой расход, чем блочное выделение по фактическому заполнению. Выгода реализации не превращает $C = 20$ в бесплатную конкурентность: полностью занятые последовательности всё равно потребуют свои KV-данные.

Измеряем на точной ревизии checkpoint, tokenizer и движка: пустой сервер; загрузка одного запроса; затем $C = 1, 4, 20$ с контролируемой длинами. Записываем пик VRAM по устройству, резервирование аллокатора, RAM, число отклонённых запросов, TTFT и время полного ответа. Калькулятор даёт гипотезу, снимок памяти — проверку. Если $C = 4$ вызывает ошибку на 6 GiB, возможны меньшая длина, другой формат кэша, offload, более компактная модель или очередь; каждое изменение заново проверяется на качестве и задержке.

Почему очередь меняет экономику памяти

Обозначим $A = 20$ пришедших запросов. При ограничении $C = 4$ остальные шестнадцать не создают полного KV для генератора, если стоят во внешней очереди. Но их пользователи ждут. Даже при одинаковых запросах сервер может эффективно объединять decode, а времена исполнения зависят от batch; поэтому нельзя вычислить p95 только делением A/C . Можно записать условный пример: если каждая четвёрка исполнялась бы ровно 12 s и накладные расходы равнялись нулю, возникло бы пять волн, последняя начиналась бы через 48 s. Это иллюстрация очереди, не прогноз сервиса. В четвёртом разделе проверим её реальными интервалами prefill и decode.

Второй результат этой темы

Оценка 6 GiB годится как ограничение прототипа с одним активным запросом и проверенной фактической памятью. Для 20 активных последовательностей она исключает заданную сырую конфигурацию без изменения режима или размещения. Даже 24 GiB следует принимать по замеру пика и сценария, а не по сумме идеальных массивов. Эта развилка определяет, что сравнивать в последующем тесте throughput и качества.

Развести два числа

A

Запросы в системе, в том числе ожидающие.

C

Активные последовательности с KV; ограничение назначает сервер.

Неизвестное

Буферы, фактическое заполнение, резервирование, другие процессы и качество после изменения формата.

Эксперимент

Один checkpoint и корпус. Прогрев, затем одинаковые длины при $C = 1, 4, 20$. Сопоставить ошибку выделения, пик, TTFT, p95 и ссылки на документ.

Адрес чтения

PagedAttention³, §2–3: причины потерь памяти. Big Model Inference²: что меняется при размещении слоёв на CPU или диске.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Hugging Face (документация, 2026), *Caching*. Документация.

Кэширование ключей и значений при генерации

Показаны варианты KV-кэша и ограничения отдельных реализаций: динамическое размещение, статический кэш и перенос. Документация помогает проверить, применяется ли простая формула полного кэша к выбранной архитектуре.

Что читать. Introduction; Cache Types; Offloaded Cache. Выписать поддерживаемый тип для конкретного checkpoint.

2. Hugging Face (документация, 2026), *Big Model Inference*. Документация.

Инференс моделей, не помещающихся целиком в ускоритель

Разъясняет dispatch весов между GPU, RAM и диском. Такая выгрузка может сделать запуск возможным, но пропускная способность пути передачи меняет практическую скорость.

Что читать. Big Model Inference и описание device map; различить факт загрузки и измеренную скорость.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Kwon, W. и др. (2023), *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP.

PagedAttention: управление памятью при обслуживании LLM

Работа вводит блочное размещение KV-кэша и показывает, как фрагментация и совместное использование влияют на пропускную способность обслуживания. Логическое число токенов и реально занятая физическая память различаются.

Что читать. §2 о проблеме памяти; §3 PagedAttention; §4 механизм vLLM и эксперименты.

4. ggml-org (документация, 2026), *Quantizing models in llama.cpp*. Документация и код.

Квантование моделей в llama.cpp

Документирует превращение GGUF-файла с весами высокой точности в конкретный тип квантования и способы оценки потерь качества. Обозначение Q4 не определяет в одиночку ни точный размер, ни применимость результата к собственной задаче.

Что читать. Вводный раздел и таблица типов квантования; сравнить размер двух файлов одного checkpoint и результат на отложенных вопросах.

Порядок чтения. Сначала Caching и Big Model Inference задают варианты хранения. Потом PagedAttention показывает, как обслуживание распределяет KV, а описание квантования позволяет проверить реальный файл. Повторить расчёт на точной конфигурации модели и измерить три режима C.

3. Шаг обучения и кластер

Обучать с нуля, полностью дообучать и менять только адаптеры — три разных нагрузки. Выбор следует из цели адаптации и измерения одного шага, а не из способности запустить инференс.

Зачем системе обновлять веса

В нашем корпусе генератор иногда неверно оформляет ссылку на страницу. Сначала сравним инструкции, retrieval и reranking на отложенной выборке: ошибка может происходить до модели. Если ошибки сохраняются, собираем пары вопрос–корректный ответ с версией документа и отдельно проверяем, не запоминает ли адаптер устаревшие факты. Дообучение оправдано только если выбранная метрика улучшается при неизменных требованиях к ссылке. Исходную и адаптированную модели сравнивают на одном наборе и одинаковом prompt.

Из чего складывается полный шаг

Идеализируем смешанную точность: параметры и градиенты по 2 bytes каждый, моменты Adam по 4 bytes каждый. Тогда $2 + 2 + 4 + 4 = 12$ bytes на параметр, то есть $8 \cdot 10^9 \cdot 12 = 96$ GB, около 89,4 GiB, только для этих четырёх массивов. Вариант с отдельной FP32 master-копией добавит ещё 32 GB; конкретная реализация может хранить иные состояния. На прямом ходе возникают активации, на обратном временные тензоры; память зависит от batch, длины, checkpointing, optimizer и распределения. Поэтому размер весов при инференсе нельзя использовать как вместимость обучения.

ZeRO³ систематизирует, какие состояния можно разделять между участниками. В FSDP2¹ параметры временно собираются для вычисления, а затем снова шардируются; при обратном ходе нужны синхронизация и временные полные тензоры. Даже если два GPU идеально делят наш 96 GB бюджет состояний до 48 GB на устройство, это не доказывает вместимости на двух 48 GB GPU: не посчитаны активации, материализация параметров и резерв. Для обучения с нуля важны также данные, число шагов и стоимость экспериментов; один успешный шаг не обещает качества модели.

Простой пробный шаг

На выделенной обучающей машине запускаем выбранный batch и длину, фиксируем память до и после forward, backward и optimizer step, время каждой стадии и восстановление после прерывания. Если пик не помещается, меняем один параметр: длину, batch, checkpointing, вариант шардирования или цель дообучения. Покупать второй GPU прежде такого опыта рано: добавится обмен между устройствами, о котором говорит NCCL².

Статический бюджет

8B, условно

Веса FP16 16 GB; градиенты 16 GB; Adam FP32 64 GB; всего 96 GB $\approx$ 89,4 GiB.

Не входят

Активации, временная материализация, рабочее место ядер, master-копия и сам корпус.

Два участника

48 GB на устройство только при идеальном делении перечисленных состояний; это нижняя оценка.

Различающая метрика

Улучшились ли ссылки на отложенных вопросах после обучения при той же версии корпуса и тем же способом поиска?

Читать

FSDP2¹: один шаг и время all-gather/reduce-scatter. ZeRO³, §2–3: различия состояний.

Адаптер вместо полного шага

Если задача допускает изменение небольшой доли параметров, QLoRA⁴ держит замороженную квантованную основу и обучает низкоранговые адаптеры. Это меняет состав оптимизируемых состояний, но не устраняет хранение основы, активаций и прочих временных тензоров. Доступный объём зависит от конкретного backend, длины и batch. Проверяем не лозунг «обучение 8B на любой карте», а замер одного шага с требуемой длиной и последующую оценку ответов. Адаптер ещё надо корректно подключить к точной ревизии основы при локальном запуске.

Что делает распределение полезным

При полном дообучении шардирование уменьшает дублирование. NCCL² предоставляет коллективные операции между GPU, но время обмена растёт с объёмом данных и зависит от фактического interconnect. Для выбора между одним большим узлом и несколькими меньшими задаём объём на шаг, площадь резервной памяти, производительность при нужном batch и время восстановления. Если обучение нужно редко, аренда проверенной конфигурации для короткого цикла может отделить эту нагрузку от постоянного сервера; это экономическое решение потребует цены и профиля занятости на дату выбора.

Протокол воспроизводимой адаптации

Сохраняем hash исходной модели и tokenizer, лицензионные условия, версию корпуса, split, параметры адаптера, число шагов, seed, runtime и артефакт результата. Один контрольный набор проверяет точность ссылок и отказ отвечать без источника; другой обнаруживает ухудшение на обычных вопросах. При обновлении документа в базе факты должны приходить из поиска, а не из заученных ответов. Если retrieval исправляет дефект дешевле и надёжнее, обучения не требуется.

Связь с выбором железа

Теперь есть два независимых требования: постоянный инференс по памяти и очереди из §2 и кратковременная адаптация по фактическому пику шага. Нельзя требовать, чтобы рабочая станция с 24 GiB выполняла наш идеализированный полный шаг. Можно проверить адаптер на ней, но только с целевой длиной и измерением. Этот вывод переносим в финальное сравнение «локальная станция плюс редкая аренда» и «собственный обучающий узел».

Дерево решения

Сначала

Проверить поиск, инструкции и качество цитат без обновления весов.

Затем

Если дефект устойчив, запустить пилот адаптера на нескольких примерах, оценить независимую выборку.

После

Если нужна полная адаптация, измерить реальный шаг и межGPU обмен; расчёт 96 GB — только начало.

Адрес чтения

QLoRA⁴, §3–4: какие тензоры обучаются. Collective Operations²: что добавляет синхронизация нескольких устройств.

Что зафиксировать

Ревизии данных, модели, адаптера; пик GPU/RAM; время шага; качество ссылок до и после.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. PyTorch (документация, 2026), *Getting Started with Fully Sharded Data Parallel (FSDP2)*. Документация.

Полное шардирование параметров, градиентов и оптимизатора

Разделяет репликацию модели и шардирование состояний обучения. Описаны коллективные операции, временное восстановление весов и компромисс между памятью и коммуникацией.

Что читать. Overview; How FSDP2 works; all-gather и reduce-scatter. Схематично изобразить один шаг на двух GPU.

2. NVIDIA (документация, 2026), *NCCL Developer Guide*. Документация.

Коллективные операции NCCL при распределённом обучении

Показывает коллективные операции и коммуникационные топологии между GPU. Для multi-GPU конфигурации скорость межсоединения и доступный набор маршрутов важны наряду с суммой VRAM.

Что читать. Overview и Collective Operations; сравнить all-reduce, all-gather и reduce-scatter для градиентов.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Rajbhandari, S. и др. (2020), *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. SC20.

ZeRO: уменьшение памяти обучения разделением состояний

Авторы разбирают память оптимизатора, градиентов и параметров и этапы их разделения между процессами. Выигрыш памяти оплачивается обменом данными и зависит от топологии кластера.

Что читать. §2 о памяти состояний; §3 этапы ZeRO; проверить, что именно шардируется на каждом этапе.

4. Dettmers, T. и др. (2023), *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS.

QLoRA: эффективное дообучение квантованных LLM

Замороженная квантованная база сочетается с обучаемыми низкоранговыми адаптерами. Работа объясняет экономию памяти и цену активаций; число параметров адаптера само по себе не определяет весь пик VRAM.

Что читать. §3 QLoRA, quantization и paged optimizers; §4 эксперименты. Восстановить состав памяти для одного шага.

Порядок чтения. Начать с FSDP2 и NCCL: построить схему одного прямого и обратного хода с обмени. Затем ZeRO объяснит классы шардируемых состояний, QLoRA — другой тип задачи. Проверить один шаг и отложенное качество, прежде чем выбирать обучающий узел.

4. Очередь и инференс

Пропускная способность токенов не описывает ожидание пользователя. Задача сервера — выбрать число активных последовательностей и выдержать требование к полному ответу при проверенном качестве.

Из чего складывается ответ

Сервер принимает вопрос, извлекает фрагменты, собирает prompt, затем выполняет prefill по входным токенам и decode по выходным. При одновременной нагрузке запрос может ждать в очереди до prefill или между этапами исполнения. vLLM Documentation¹ показывает устройство сервера и параметры планирования; PagedAttention³ объясняет выделение KV в блоках для переменных длин. Для нашего $A = 20$ сначала проверим $C = 1, 4$ и, если память позволяет, 20, удерживая одинаковыми корпус и checkpoint.

Пользователю важны время до первого токена (TTFT), промежутки между последующими токенами (TROT), время до полного ответа и доля запросов с верной ссылкой. Модель может выдавать высокий aggregate tokens/s и одновременно заставлять поздние запросы долго ждать. Benchmark CLI² задаёт определения серверных метрик; вместе с ними сохраняем распределение входных и выходных длин, rate поступления и число ошибок. «p95 меньше 120 s» означает, что не менее 95% измеренных полных ответов в зафиксированной нагрузке уложились в предел; одного среднего недостаточно.

Первая развилка: четыре активных ответа

Ранее $C = 4$ потребовал нижнюю сумму 5,73 GiB; на 6 GiB осталось лишь 0,27 GiB для всего остального. Пилот на такой VRAM может не пройти даже загрузку. Если реальный пик укладывается только при $C = 1$, двадцать поступивших запросов образуют очередь; это ограничение полезно показать пользователям, а не скрывать за метрикой одиночного запроса. На условной 24 GiB карте $C = 20$ оставляет 10,27 GiB до накладных расходов, но увеличение активного batch может ухудшить задержку отдельного пользователя. Эту гипотезу проверяет нагрузочный опыт.

План воспроизводимого теста

Сначала на фиксированных длинах сравниваем $C = 1$ и 4, прогрев отдельно. Затем формируем двадцать запросов с коротким и длинным prompt и регистрируем время прихода, начало prefill, первый и последний токен, пик памяти и ошибку цитирования. Пробуем $C = 20$ только там, где позволяет измеренная память. Если очередь растёт, меняем admission limit, форму batch или число экземпляров, снова проверяя качество.

Четыре разные меры

TTFT

Приход запроса → первый токен, включая ожидание, если так задан измеритель.

TROT

Интервал последующих токенов.

Полный ответ

Приход → последний токен плюс требуемая проверка ссылки.

Goodput

Ответы, удовлетворившие заданным задержке и качеству, за единицу времени.

Постоянные опыты

Checkpoint, tokenizer, корпус и его версия, входные и выходные длины, код сервера и политика очереди.

Читать

Benchmark Serving²: определения и параметры генератора нагрузки. Serving¹: возможности планировщика.

Когда ускорение prefill не ускоряет весь сервис

Представим, что длинные документные запросы занимают устройство при обработке префикса, мешая уже начатым decode. DistServe⁴ исследует разделение двух стадий и оптимизацию goodput при ограничениях TTFT/TPOT. Это специальная архитектура, у которой есть стоимость переноса KV и дополнительные узлы. Если на нашей небольшой нагрузке доминирует время поиска, разделение генератора ничего не исправит; если большие префиксы систематически задерживают decode, ставим опыт с раздельным планированием прежде масштабирования узлов.

Сколько запросов выдержит один узел

Для простого контроля применяем не «токенов в секунду в одиночку», а тест при последовательных rate поступления. При каждом rate фиксируем p_{50} , p_{95} полного ответа и TTFT, throughput, процент отказов и точность цитирования. Допустимой называем лишь область, где $\Pr(T_{\text{complete}} \leq 120 \text{ s}) \geq 0,95$ на нашей репрезентативной выборке и качество ссылок прошло заранее определённый порог. Здесь 120 s и 95% — требования учебного проекта; измерений пока нет. Если нагрузка выше этой области, снижать C ради памяти может повысить ожидание; повышать C без VRAM нельзя. Альтернативы: меньше окно при сохранении доказательств, другая модель, дополнительный экземпляр или узел.

Продуктовое следствие

Для одного исследователя достаточно проверенного режима $C = 1$ с понятным временем. Для пятнадцати пользователей закупка оправдывается протоколом одновременной нагрузки: устройство, которое только запускает модель, может не выдержать SLA. Если обслуживающая система хранит корпус на другом узле, проверяем сетевое время и ответственность при отказе. Сервер должен возвращать и происхождение цитаты; ускорение ответов без корректной версии документа не считается выполнением задачи.

Передача к данным

Время поиска и достоверность ссылки мы пока держали постоянными. В действительности index и база имеют отдельные задержки, ёмкость и восстановление. Следующий раздел покажет, почему ускорение векторов нельзя оценивать без версии исходного документа и проверки результата на точном поиске.

Проверка режима C

Одна машина

Те же двадцать запросов; сравнить $C = 1, 4$, а на достаточной VRAM также 20.

Запись

Время прихода и ожидания, начало prefill, первый/последний токен, p_{95} , ошибки, пик VRAM.

Решение

Выбрать наибольшую проверенную нагрузку с заданным временем и качеством, не только максимальный tokens/s.

Гипотеза фронта

DistServe⁴, §2–4: помехи prefill/decode и цена переноса KV; проверить, есть ли такая помеха у нас.

Ограничение

Условные 5 волн по 12 s из §2 не являются оценкой p_{95} : batch меняет сервисное время.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. vLLM (документация, 2026), *vLLM Documentation*. Документация и код.

Сервер инференса vLLM и его планировщик

Объединяет планирование запросов, управление KV-памятью и запуск сервера. Конкретные параметры зависят от версии; исходный runtime необходимо фиксировать вместе с измерениями.

Что читать. *Serving* и *Configuration*; проследить вход запроса до выделения KV-блоков и ответа.

2. vLLM (документация, 2026), *Benchmark CLI*. Документация.

Измерение времени первого токена и времени между токенами

Документирует параметры генератора нагрузки и метрики TTFT, TPOT, throughput и задержки. Сопоставление машин требует одинаковой модели, распределения длин и интенсивности поступления запросов.

Что читать. *Benchmark Serving* и определения метрик; составить команду опыта с воспроизводимыми входными длинами.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Kwon, W. и др. (2023), *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP.

PagedAttention: управление памятью при обслуживании LLM

Работа вводит блочное размещение KV-кэша и показывает, как фрагментация и совместное использование влияют на пропускную способность обслуживания. Логическое число токенов и реально занятая физическая память различаются.

Что читать. §2 о проблеме памяти; §3 PagedAttention; §4 механизм vLLM и эксперименты.

ФРОНТИР ИССЛЕДОВАНИЙ

4. Zhong, Y. и др. (2024), *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI.

DistServe: раздельное исполнение prefill и decode

Предлагается разместить фазы prefill и decode на разных группах GPU, чтобы уменьшить помехи и отдельно выбирать их параллелизм. Передача KV-состояния тоже имеет цену, поэтому результат зависит от нагрузки и сети.

Что читать. §2 анализ помех; §3 архитектура; §4 оптимизация размещения и измерения goodput.

Порядок чтения. Сначала vLLM и Benchmark CLI дают контроль исполнения и измеритель. PagedAttention объясняет роль KV-памяти. DistServe имеет смысл после профиля, показавшего помехи между prefill и decode. Итог — допустимая область поступления запросов при своей метрике полного ответа.

5. Корпус, индекс и копии

Хранение ответа включает исходный документ, его версию, фрагменты и индекс. Ускорение поиска полезно, только если возвращённую цитату можно проверить и восстановить после сбоя.

Векторы — небольшая часть системы

Для учебных 150 тысяч фрагментов и размерности эмбединга 1024 сырые float32-векторы занимают $150\,000 \cdot 1024 \cdot 4 = 614\,400\,000$ bytes, около 0,572 GiB. Это число не включает исходные PDF и таблицы, текст фрагментов, метаданные страниц и ячеек, индекс, служебные структуры БД, WAL и резервные копии. При другой модели эмбедингов изменятся размерность и качество поиска; миграция требует версии вектора и пересчёта. Оценка SSD по одной таблице векторов дала бы неверное решение.

Линия происхождения ответа

Пусть PDF v2 содержит формулировку на странице 7. Храним immutable-идентификатор файла или hash, версию обработки, координату, текст фрагмента, версию embedding и момент индексации. Запрос сначала применяет контроль доступа и фильтры, затем ищет кандидатов, возможно пересортировывает, передаёт выдержку модели и в итоговом ответе ссылается на PDF v2, с. 7. После выхода PDF v3 старую ссылку нельзя молча подменять новой: ответ должен указывать ту версию, по которой построен. Introduction to Information Retrieval¹ даёт основы индекса и оценки ранжирования; проверять надо и релевантность, и происхождение.

Точный и приближённый поиск

pgvector² поддерживает точный поиск и приближённые индексы, в том числе HNSW. При фильтрации по проекту, правам доступа или версии документов выдача ANN может иметь другую полноту, чем точный отбор; настройки поиска меняют задержку и recall. HNSW³ показывает алгоритм и стоимость индекса. Собираем набор вопросов с правильными фрагментами, сначала считаем точную выдачу после корректных фильтров, затем сопоставляем ANN при том же наборе и измеряем p95 поиска, recall@k и число неверных ссылок. Если точного эталона нет, один красивый ответ не доказывает качество индекса.

Узел хранения

Для базы важны RAM под рабочий набор, CPU для фильтров и параллельного поиска, SSD для random I/O, WAL, обновлений и резервирования, сеть для общения с генератором. «Самый быстрый GPU» не компенсирует ошибку прав доступа или потерю версии. Объём RAM и SSD берём из реально построенного индекса и суточного прироста, добавляя запас на перестроение и копию.

Число и границы

Сырые векторы

$150k \times 1024 \times 4 \text{ bytes} = 614,4 \text{ MB} \approx 0,572 \text{ GiB}$.

Отдельно

Документы, OCR, фрагменты, метаданные, HNSW, WAL, резервная копия, временное перестроение.

Цитата

ID файла → версия v2 → страница/ячейка → фрагмент → ответ. Любое звено проверяется по источнику.

Адрес чтения

Introduction to IR¹, гл. 1, 6, 8: индекс и метрики. Filtering²: порядок фильтров и влияние ANN.

Обновление без тихой подмены доказательства

При новой версии PDF парсер может изменить границы фрагментов и координаты. Записываем её отдельно, строим новый индекс и све-ряем поисковые тесты прежде переключения. Метаданные должны позволять вернуть страницу оригинала даже после переиндексации. Embedding и генератор имеют разные версии и сроки обновления; их миграции не обязаны совпадать. Практика выборки нескольких отве-тов с переходом по цитате обнаруживает ошибки, которых не увидит измерение только vector recall.

Отказ и восстановление

PostgreSQL PITR⁴ описывает связку базовой копии и архива WAL для восстановления до выбранной точки. Наличие бэкапа без пробного восстановления не доказывает, что текст и версия документа доступ-ны. На отдельном узле восстанавливаем базу, исходные файлы или их согласованное хранилище, затем запускаем сохранённый вопрос к PDF v2. Фиксируем RPO — допустимую потерю последних изменений и RTO — допустимое время восстановления. Если архив WAL и доку-менты живут независимо, согласованность версии придётся прове-рять явно.

Проверка стоимости поиска

Измеряем p95 retrieval при пятнадцати пользователях и обновлении документов, объём RAM после прогрева, дисковый рост за неделю, время перестроения и восстановления. Только после этого выбираем отдельную базовую машину, диск и сеть. Если retrieval мал по сравне-нию с генерацией и помещается в RAM, GPU сервера может оказать-ся приоритетнее; если база выдаёт устаревшую версию, увеличение GPU вообще не решает дефект. Эти исходы направляют финальную конфигурацию.

Переход к рабочей станции

Та же документная система может быть личным исследовательским инструментом без сервера на пятнадцать пользователей. Локальный пользователь всё равно обязан хранить версии и проверять ссылки, но конкурентность и соглашение о времени другие. Следующая часть разберёт путь от небольшой локальной модели до более крупной: что в действительности подтверждает совместимость, а что лишь выгля-дит как подходящая видеокарта.

Проверка после сбоя

Снять

Копию базы, метаданных, документа и необходимого WAL в согласованных версиях.

Восстановить

На отдельной машине до указанного момента.

Убедиться

По старому вопросу открыть PDF v2, страницу 7, а не «похожий» фрагмент v3.

Результат для железа

Требуемые RAM/SSD и сеть получаются из размера реального индекса, пика обновления и времени восстановления.

Читать

HNSW³, §3–4: обмен recall и стоимости. PITR⁴: архив WAL, база и восстановление.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Manning, C. D.; Raghavan, P.; Schütze, H. (2008), *Introduction to Information Retrieval*. Учебник.
Введение в информационный поиск
Объясняет инвертированные индексы, ранжирование и оценку поиска. Качество RAG начинается с выбора релевантных фрагментов, а не с объёма VRAM генератора.
Что читать. Главы 1, 6 и 8: индекс, ранжирование и оценка; задать тестовый набор запросов.
2. pgvector (документация, 2026), *pgvector: Open-source vector similarity search for Postgres*. Документация и код.
Векторный поиск в PostgreSQL
Описаны точный поиск, HNSW и IVFFlat, типы векторов, индексы и SQL-фильтрация. Выбор индекса отражается на RAM, размере диска, времени построения и качестве поиска.
Что читать. Exact Search; HNSW; IVFFlat; Filtering. Замерить recall относительно точной выдачи после фильтра.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Malkov, Y. A.; Yashunin, D. A. (2016), *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*. Исследовательская статья.
HNSW: приближённый поиск по иерархическому графу близости
Иерархический граф позволяет быстро находить кандидатов, меняя время и память на достигаемую полноту. Векторный индекс хранит связи между точками помимо самих чисел вектора.
Что читать. §3 построение и поиск; §4 эксперименты. Сравнить стоимость индекса и recall с точным поиском.
 4. PostgreSQL Global Development Group (документация, 2026), *Continuous Archiving and Point-in-Time Recovery (PITR)*. Документация.
Непрерывное архивирование и восстановление базы на момент времени
Резервная копия и журнал WAL позволяют восстановить согласованное состояние. Для RAG надо также согласовать версию исходного документа, фрагментов, эмбедингов и индекса.
Что читать. Setting Up WAL Archiving; Making a Base Backup; Recovering. Провести пробное восстановление на отдельном узле.
- Порядок чтения*. Начать с IR и точного baseline на собственных запросах. Затем читать pgvector и HNSW и сравнить ANN с baseline под фильтрами. PITR завершает цикл: восстановить базу и исходный документ, убедиться, что старые цитаты открываются.

6. Локальная модель: от SLM до LLM

У локального запуска есть проверяемая цепь совместимости: конкретный checkpoint, формат, tokenizer, движок, операционная система и устройство. Память определяет возможные режимы, опыт — их качество и скорость.

Выбрать задачу, затем масштаб

Для личного вопроса по нескольким документам сначала проверяем небольшую модель, условно класса SLM, на отдельной выборке вопросов с точными ссылками. Более крупная LLM нужна только если прирост качества оправдывает расход памяти и время. Число параметров не говорит, насколько модель понимает таблицы, русский текст или верно отказывается при отсутствии источника. Model Cards¹ помогает выяснить происхождение и условия использования, но сама карточка не заменяет замер. В протокол вносим точный идентификатор и ревизию весов, tokenizer, chat template, лицензию, формат и результаты на своих данных.

Локальный контур включает парсер/OCR, индекс, embedding, возможно reranker, генератор и интерфейс. Можно запускать их последовательно на одном устройстве, а можно одновременно. Первый режим экономит память ценой переключения и задержки; второй требует считать сумму пиков. Поэтому вопрос «запустится ли 8B?» не равен «поместится ли весь локальный RAG?». У пользовательской машины часть RAM займут ОС, браузер, исходные файлы и фоновые приложения.

Конкретный путь CPU и GPU

Plama.cpp² позволяет проверить выбранный файл на CPU, полностью на поддерживаемом GPU или с частичным размещением слоёв. Записываем не только факт успешной загрузки, но и фактическое размещение, пик RAM/VRAM, размер prompt, TTFT и устойчивую выдачу после прогрева. Если веса или KV выходят за VRAM, offload возможен, но измерение решит, приемлема ли задержка. Для SLM рабочий CPU с достаточной RAM иногда достаточен для личных запросов; для LLM с длинными входами важен одновременно размер весов, KV и bandwidth. Не выводим нужные гигабайты CPU-RAM из гигабайтов VRAM один к одному.

Проверка качества формата

Для одного checkpoint сравниваем исходный доступный формат и две версии квантования на одинаковых документах: точная цитата, отказ при отсутствии ответа, неправильная версия, время и память. Наш идеальный Q4=3,73 GiB из §2 не задаёт размер фактического GGUF и ничего не доказывает про качество. Отдельно проверяем допустимую длину контекста и реальное KV после составления RAG prompt.

Паспорт локального запуска

Артефакт

Модель, ревизия, tokenizer, template, лицензия, формат весов.

Стек

Версии ОС, драйвера, backend и приложения.

Нагрузка

Длина prompt/output, один или несколько активных ответов; embedding и reranker вместе или по очереди.

Наблюдение

Загрузка, пик памяти, TTFT, полный ответ и верная ссылка.

Адрес чтения

Model Cards¹: ограничения модели. README/server²: конкретный локальный запуск и параметры.

Матрицы совместимости вместо догадки по бренду

Для дискретной карты проверяем точную модель GPU и поддерживаемый backend нужного движка. ROCm Compatibility Matrix³ называет сочетания GPU, ОС и версий стека; у NVIDIA аналогично проверяют compute capability, драйвер, CUDA и требуемые kernels. Поддержка «семейства» не доказывает работу конкретного формата или версии приложения. Для Apple Silicon MLX⁴ предоставляет собственный путь выполнения; общая память GPU и CPU остаётся ограниченным общим ресурсом. Две системы с одинаковым объявленным объёмом памяти имеют разные пути передачи и требования к ПО.

Заполняем таблицу «точный checkpoint × формат × движок × устройство × версия стека» и ставим отметку только после запуска. При невозможности загрузить нужный tensor type, ограничении длины или ухудшении качества вариант исключается. CPU fallback полезен для доступности, но его время проверяется отдельно; шильдик GPU не заменяет скорость законченного ответа.

Если требуется адаптация

Локальное дообучение адаптера — отдельный профиль из §3: датасет, длина, batch, обучение шага и сохранение. После него важно проверить, что адаптер относится к той же ревизии базовой модели и воспроизводимо запускается в выбранном inference backend. Если нужный runtime для обучения и запуска различается, закладываем проверяемый путь конвертации; размер VRAM не решает несовместимость форматов.

Два локальных исхода

На имеющихся 6 GiB сначала испытываем SLM и выбранную 8B при $C = 1$, фиксируя фактический пик и скорость; рассчитывать на режим $C = 4$ по нижней границе нельзя. На гипотетической 24 GiB станции испытываем ту же модель с $C = 1, 4$, при необходимости двадцать активных запросов, но многопользовательское обслуживание оцениваем отдельно от личной работы. Если маленькая модель проходит тест цитат и времени, увеличение модели и карты не требуется для этой задачи. Если нет — сравниваем более крупный checkpoint, дополнительные стадии поиска и доступный удалённый сервис по одной оценке.

Четыре проверки

Совместимость

Точный GPU/CPU, ОС, драйвер, backend, формат.

Вместимость

Пик RAM/VRAM при нужной длине и соседних процессах.

Скорость

TTFT и полный ответ после прогрева.

Смысл

Отложенные вопросы, страница/ячейка, отказ при отсутствии источника.

Читать

Compatibility Matrix³: сверить комбинацию версий. README⁴: общая память и путь исполнения на Apple Silicon.

Фиксация

Сохранить команду и конфигурацию запуска. «Поддерживается» становится фактом лишь после конкретного опыта.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Hugging Face (документация, 2026), *Model Cards*. Документация.

Карточки моделей: лицензия, назначение и ограничения

Карточка связывает веса с задачей, происхождением, условиями использования и известными ограничениями. Это начало проверки, а не измерение пригодности на собственных документах. Для воспроизводимого запуска отдельно зафиксируйте ревизию checkpoint, tokenizer, шаблон диалога и лицензию.

Что читать. Overview и metadata; прочитать карточку выбранной ревизии, затем сверить конфигурацию tokenizer и провести задачу на отложенных примерах.

2. ggml-org (документация, 2026), *llama.cpp*. Документация и код.

llama.cpp: локальный запуск и сервер моделей

Проект показывает локальный запуск CPU/GPU, частичную выгрузку слоёв, сервер и параметры контекста. Поддержка backend не гарантирует, что точный checkpoint корректно загрузится или достигнет нужной скорости. Запишите команду, ревизию весов и число фактически выгруженных на GPU слоёв.

Что читать. README, build instructions и server README; запустить выбранный checkpoint на CPU и GPU, сравнить размещение, первый токен и устойчивую генерацию.

3. AMD (документация, 2026), *ROCm Compatibility Matrix*. Документация.

Матрица совместимости ROCm

Матрица сопоставляет конкретные устройства, ОС, версии ROCm и поддерживаемые фреймворки. Проверка только серии GPU оставляет неопределёнными сборку runtime и требуемые операции модели. Сохраните выбранную строку матрицы рядом с протоколом фактического запуска.

Что читать. GPU, OS and framework support именно своей версии; отметить подтверждённые и неизвестные сочетания до установки стека.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

4. Apple ML Explore (документация, 2026), *MLX*. Документация и код.

MLX: вычисления на Apple Silicon

Описывает стек для CPU и GPU Apple Silicon с общей памятью и собственным API. Общая память облегчает обмен данными, но приложения и система делят конечную ёмкость; bandwidth и тепловой режим также ограничивают длительную выдачу. Указание поддержки MLX само по себе не проверяет конкретную модель.

Что читать. README и документация о device/array; проверить формат выбранных весов, пик общей памяти и скорость генерации после прогрева.

Порядок чтения. Model Cards задают ограничения и ревизию артефакта. Потом llama.cpp даёт опыт CPU/GPU; ROCm Matrix и MLX помогают развести несовместимые платформы. Итог — таблица реально проверенных режимов SLM и LLM, а не список брендов.

7. Выбор и проверка машины

Финальная конфигурация выводится из пика памяти, задержки полного ответа, качества ссылки и частоты обучения. Ни один компонент не выбирают по максимальному паспортному числу в отрыве от этих четырёх проверок.

Три конфигурации для одного проекта

Вариант А — существующая машина с 6 GiB GPU: личный прототип, $C = 1$ при успешной загрузке, SLM или проверенная 8B с допустимым размещением, корпус и база локально. Вариант Б — рабочая станция с условными 24 GiB: тот же checkpoint и больше проверенных активных запросов, но без гарантии $C = 20$ или времени 120 s. Вариант В — отдельный обслуживающий узел плюс база, а обучение адаптера или полный шаг выполняется на арендованном обучающем узле, если измерения подтвердили необходимость. Это классы решения, не готовый список SKU.

Как выбирать компоненты по причинной цепи

Сначала GPU: достаточная фактическая VRAM для весов, KV и пика runtime; подтверждённые форматы, архитектура и backend; измеренные prefill/decode и длительная мощность. Compute Capability¹ помогает проверить возможности точного чипа. Затем CPU: парсер/OCR, токенизация, поиск и серверные потоки, измеренные по трассе. RAM покрывает индекс, файловый кэш, offload и соседние процессы с запасом. SSD — исходники, модели, индекс, WAL, перестроение и копии, с проверкой случайного доступа и восстановления. Сеть важна при разнесении базы и генератора; для распределённого обучения важен ещё обмен между GPU. Материнская плата должна иметь подходящие слоты/линии, поддержку RAM и интерфейсов накопителей; блок питания и охлаждение рассчитывают по полной длительной нагрузке, разёмам и запасу производителя, а не только TDP ускорителя.

Что исключает расчёт, а что лишь допускает

Идеальные веса и полный KV при $C = 20$ дают 13,73 GiB: выбранная конфигурация 6 GiB исключается для указанного размещения без изменения режима. 24 GiB этим *не подтверждены*: остаются runtime, соседние модели, другие длины и нагрев. Полный обучающий шаг 96 GB состояний исключает трактовку локального 24 GiB как машины для указанного полного обновления без радикально иного размещения. Адаптер надо замерять. Данные и цитаты имеют собственный критерий, не выражаемый в GB GPU.

Лист сравнения

Для каждой машины заполняем одну строку: цена приобретения/аренды на дату решения; стоимость энергии и обслуживания; совместимость; пик GPU/RAM/SSD; p95 полного ответа при одинаковом потоке; доля правильных ссылок; время восстановления; результат пилота обучения. Пока этих чисел нет, выбор класса есть гипотеза, а не рекомендация конкретной покупки.

Фильтр покупки

Нельзя

Точная модель или backend не запускаются; память не выдерживает нужное C ; нарушено качество ссылок.

Можно испытывать

Есть запас по нижней оценке, подтвердился стек и корпус, известны реальные пики.

Принимать

Проверены p95, цитаты, длительное исполнение и восстановление на своём потоке.

Сверка

Compute Capability¹: архитектура чипа. Memory Usage³: расхождение нижней оценки и пика.

Учебный бюджет RAM

Одновременно

ОС и приложения 6 GB + индекс 2 GB + OCR 4 GB + offload 4 GB = 16 GB. С запасом 25%: 20 GB; испытывать класс 32 GB RAM, если всё действительно работает вместе.

Учебный бюджет SSD

Хранить

Документы 20 GB + два checkpoint 12 GB + индекс 5 GB + WAL 10 GB + копия 25 GB + перестроение 5 GB = 77 GB. Запас 30%: около 100 GB; рост и ресурс записи не включены.

Парный опыт перед решением

На кандидатах А и Б запускаем одинаковую ревизию модели и корпуса, тот же runtime, prompt и поток двадцати запросов. Разрешаем каждой машине свою разумную настройку C , но записываем её как часть конфигурации; отдельный контроль фиксирует одинаковое C . Сравниваем статистику по тем же запросам после прогрева, полное распределение времени, пик памяти и проверку ссылки. Для иллюстрации: четыре условных полных времени А равны 12, 10, 14, 12 s, Б — 8, 9, 10, 9 s; средняя разница — 3 s. Это арифметический пример, не измерение и не аргумент о р95: четыре точки и иной порядок нагрузки не дают уровня сервиса.

PyTorch Profiler² и серверные трассы помогают локализовать причину разницы; Understanding CUDA Memory Usage³ показывает тензоры и allocator, но память сторонних библиотек может оказаться вне снимка. Отмечаем внешние аллокации по устройству. Результаты MLPerf Inference⁴ можно читать как внешний ориентир для определённого сценария, не как обещание RAG с нашей моделью и документами.

Вердикт учебного проекта

Начать с А как доступного прототипа и проверить SLM, затем 8B в режиме $C = 1$; обработка корпуса и цитирование должны работать отдельно от выбора генератора. Если прототип не выдержал качество, попробовать другой checkpoint или поиск. Если выдержал качество, но не нагрузку, проверить Б при $C = 4, 20$ и $p95 \leq 120$ s; при неуспехе разделить сервис или изменить требования. Обучение проверять на отдельном пилоте и, если оно редкое, сравнить аренду с покупкой по фактическим часам. До протокола нельзя честно сказать пользователю «бери такую-то карту».

Что осталось измерить

Точная ревизия реальной модели, распределение длин запросов, доля подходящих фрагментов, запросы по доступам, объём исходных документов, рост корпуса, часы адаптации, тарифы и фактический пик на машинах. После заполнения этих полей заменяем условные примеры отчётом, пересчитываем память и фиксируем принятое решение вместе с датой. Именно этот повторяемый метод должен пережить смену поколения GPU.

Минимальный протокол

Одинаковый вход

Вопросы, ревизии данных, checkpoint, tokenizer, длины и тест ссылок.

Изменение

Устройство и настройка C ; записать оба.

Результат

Распределение времени, пик, ошибки, качество и стоимость.

Что читать

Profiler²: разобрать трейс стадий. MLPerf⁴: проверить совпадение сценария прежде цитирования цифр.

Состояние решения

6 GiB — проверить прототип; 24 GiB — кандидат на опыт обслуживания; полный шаг обучения — отдельная машина. Конкретную покупку определяют измерения.

Учебный бюджет питания

Оценка

GPU 250 W + CPU 125 W + остальная система 60 W = 435 W; запас 30% даёт не менее 566 W номинала. Проверить разъёмы, кратковременные пики и длительную нагрузку. Это не расчёт годовых затрат на энергию.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. NVIDIA (документация, 2026), *CUDA GPU Compute Capability*. Документация.

Вычислительные возможности GPU NVIDIA

Возможности чипа связаны с набором операций и поддержкой численных форматов. Два ускорителя с одинаковой VRAM могут по-разному исполнять нужные ядра. Соответствие чипа таблице ещё не проверяет версии драйвера, сборку приложения и конкретный checkpoint.

Что читать. Таблицы compute capability и feature support; проверить точную архитектуру GPU, затем подтвердить её пробным запуском нужного runtime.

2. PyTorch (документация, 2026), *PyTorch Profiler*. Документация.

Профилирование вычислений и памяти в PyTorch

Профайлер показывает время операций, активность CPU/GPU и часть выделений памяти. Он позволяет проверить гипотезу об узком месте, например сравнить токенизацию с исполнением ядер, но собственные метрики сервера и поиск надо измерять отдельно.

Что читать. Profiler API и trace; измерить одинаковые запросы после прогрева, сопоставить CPU с GPU, затем добавить серверные TTFT и p95.

3. PyTorch (документация, 2026), *Understanding CUDA Memory Usage*. Документация.

Понимание расхода памяти CUDA

Snapshot и статистика allocator позволяют отделить память активных тензоров, резерв и пик во времени. Это помогает понять, почему нижняя оценка весов и KV не равна занятой VRAM. Аллокации вне видимости PyTorch надо учитывать иными средствами.

Что читать. Recording memory history и Visualizing snapshots; отметить, какие расходы инструмент не видит, и сравнить пик с расчётом главы.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

4. MLCommons (документация, 2026), *MLPerf Inference*. Бенчмарк и правила.

MLPerf Inference: сценарии и сопоставимые измерения

Набор задаёт сценарии, пороги точности и правила сравнения систем. Публичная цифра throughput полезна как внешний ориентир только после сверки модели, входных данных, ограничений задержки и настроек. Для закупки по документному RAG нужен собственный поток запросов и проверка качества ссылок.

Что читать. Scenarios, rules and results; выписать несовпадения со своей нагрузкой и разработать парный опыт на точном checkpoint.

Порядок чтения. Compute Capability и Memory Usage закрывают совместимость и фактический пик. Profiler устанавливает причинность задержки, MLPerf учит читать границы внешнего сравнения. После чтения провести парный опыт, заполнить лист выбора и сохранить решение с датой.